GymManager Pro

Manual Tecnico

Proyecto:	PIA - Programacion Orientada a Objetos
Tecnologia:	Java 21 + JavaFX 21 + Maven
Autor:	Elias Jacob Bernal
Fecha:	Mayo 2026

1. Descripcion general del sistema

GymManager Pro es una aplicacion de escritorio para la administracion de un gimnasio. Permite gestionar clientes, membresias, clases grupales, equipos, pagos y accesos de entrada/salida, todo desde una interfaz grafica con modo claro y oscuro.

Modulo	Descripcion
Clientes	Alta, baja, modificacion y busqueda de miembros
Membresias	Asignacion de planes, renovacion automatica
Clases grupales	Horarios, instructores, inscripcion de clientes
Equipos	Inventario con estado de mantenimiento
Pagos	Registro de cobros con diferentes metodos
Accesos	Control de entrada y salida en tiempo real
Reportes	Generacion de PDF con resumen del sistema

Tecnologias utilizadas

- Java 21 con records, streams y var
- JavaFX 21 para la interfaz grafica
- Maven como sistema de construccion
- Serializacion Java para persistencia local
- iText / OpenPDF para generacion de reportes PDF

2. Arquitectura del proyecto (patron MVC)

El sistema sigue el patron MVC (Model-View-Controller). Cada capa tiene una responsabilidad especifica y no se mezclan entre si: la Vista construye la UI, el Controller valida y coordina, y el Model junto con los servicios gestionan los datos.

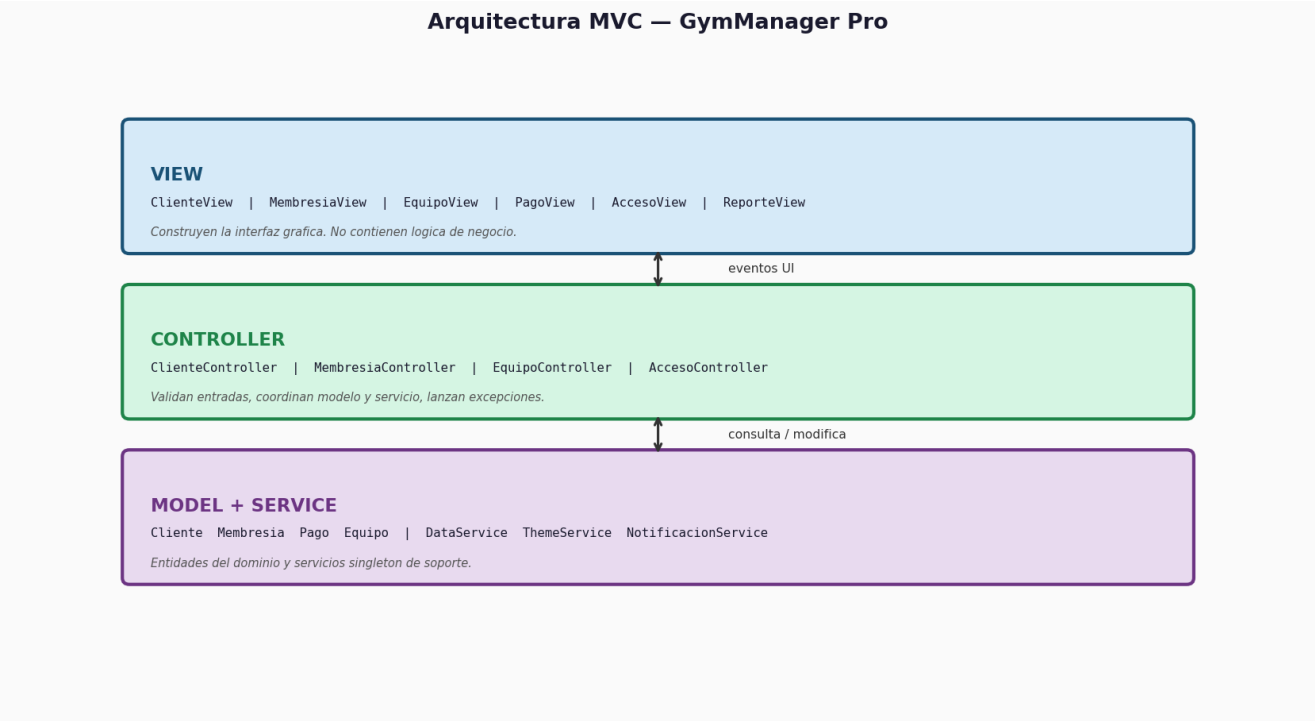

Figura 1 - Arquitectura MVC de GymManager Pro

3. Diagrama de paquetes

El proyecto se organiza en 8 paquetes con responsabilidades claras. Las flechas indican dependencias: view usa controller, controller usa model y service.

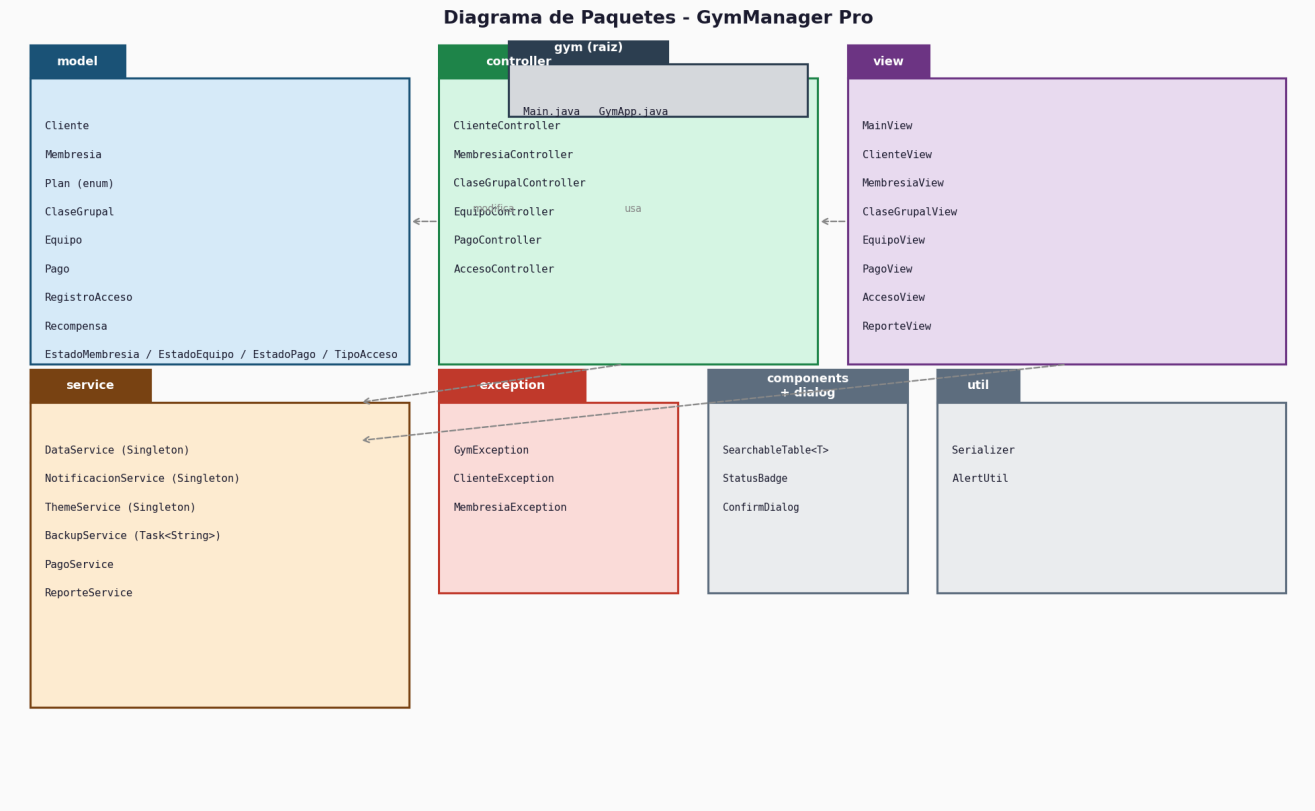

Figura 2 - Diagrama de paquetes de GymManager Pro

4. Modelo de dominio - Diagrama de clases

Las entidades del dominio implementan Serializable para poder persistirse en disco. Cada clase administra su propio contador de IDs estatico para garantizar unicidad.

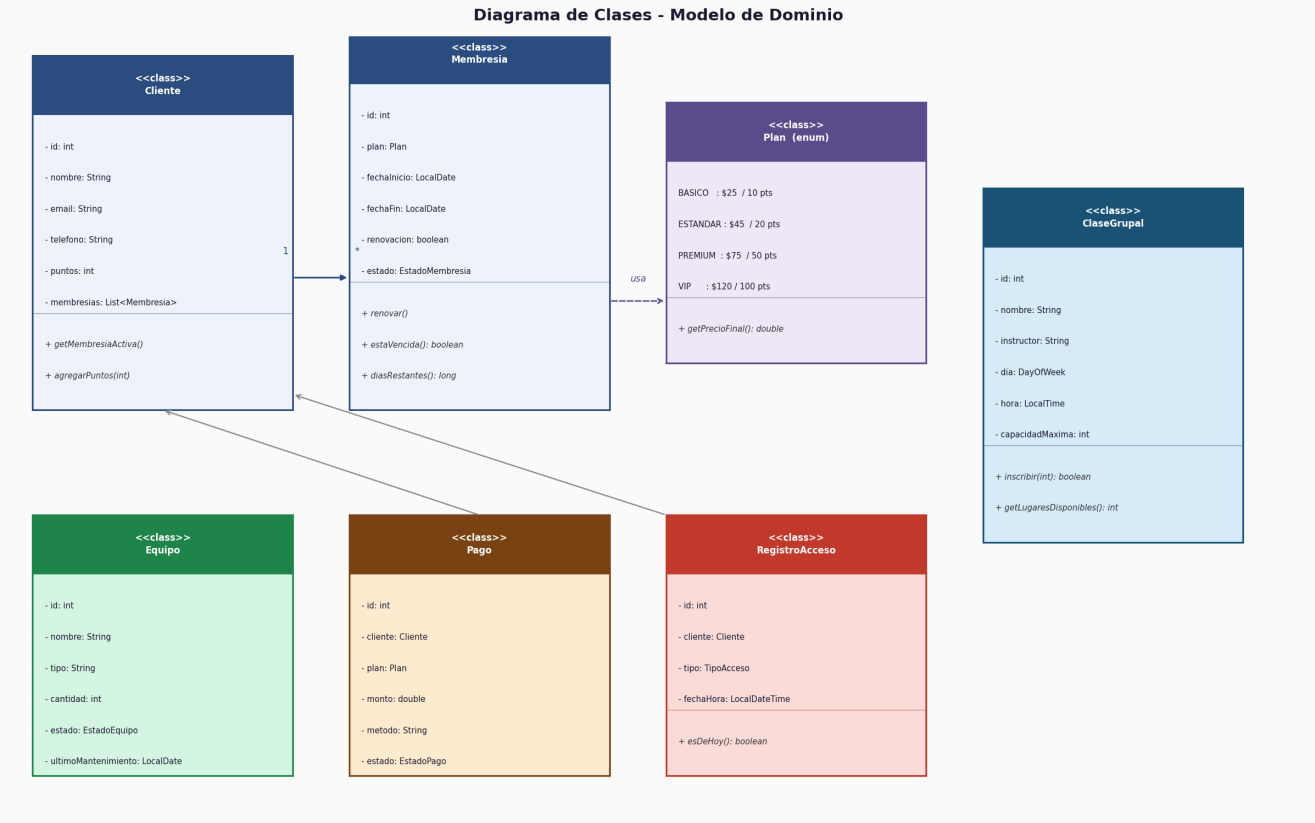

Figura 2 - Diagrama de clases del modelo de dominio

4. Modelo de dominio (continuacion) - Enumeraciones

El sistema usa enumeraciones para representar estados y categorias fijas del dominio, evitando el uso de Strings magicos y aprovechando el tipado fuerte de Java.

Enum	Valores
Plan	BASICO ($25/10pts), ESTANDAR ($45/20pts), PREMIUM ($75/50pts), VIP ($120/100pts)
EstadoMembresia	ACTIVA, VENCIDA, SUSPENDIDA
EstadoEquipo	DISPONIBLE, EN_MANTENIMIENTO, FUERA_DE_SERVICIO
EstadoPago	PENDIENTE, COMPLETADO, RECHAZADO
TipoAcceso	ENTRADA, SALIDA

Plan (enum) con logica de negocio

Plan es un enum especial que contiene atributos (precio, descuento, puntosBonus) y un metodo getPrecioFinal() que aplica el descuento correspondiente a cada nivel.

```
  BASICO  ("Basico",   25.0,  0.00,  10)
ESTANDAR("Estandar", 45.0,  0.05,  20)   // 5% descuento
PREMIUM ("Premium",  75.0,  0.10,  50)   // 10% descuento
VIP     ("VIP",     120.0,  0.20, 100)   // 20% descuento

double getPrecioFinal() { return precio * (1 - descuento); }
```

5. Capa de servicios

Los servicios son clases Singleton que proveen funcionalidad transversal. Ninguna vista o controlador crea instancias directamente; siempre usan getInstance().

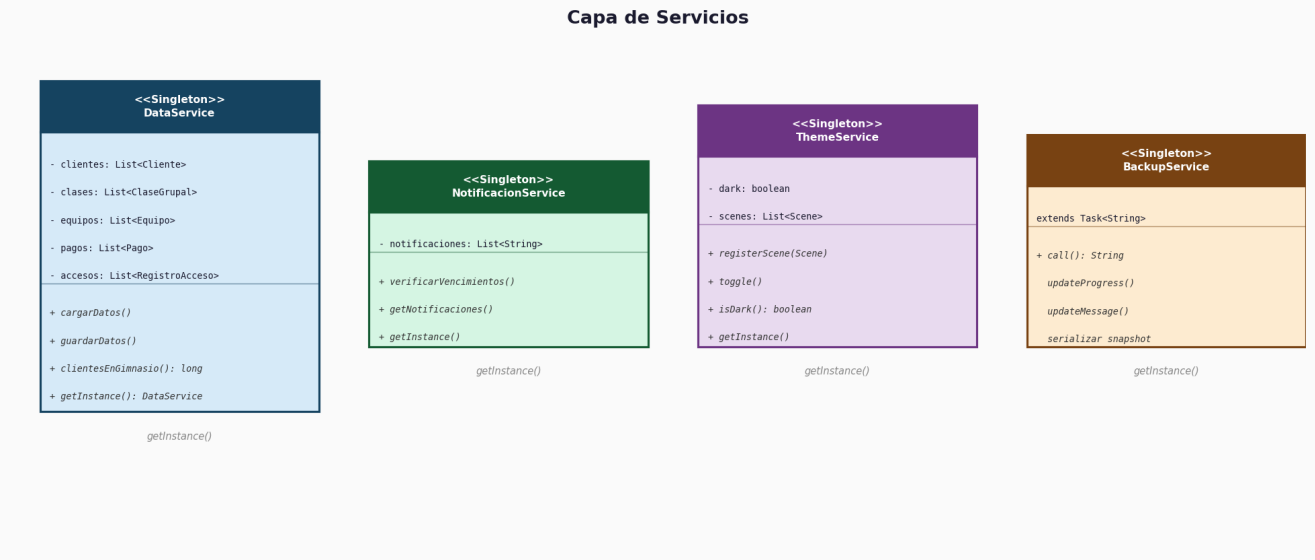

Figura 3 - Servicios Singleton del sistema

6. Jerarquia de excepciones

Se definio una jerarquia propia para representar errores del dominio del gimnasio. Todas son checked exceptions: el compilador obliga a capturarlas o declararlas en cada punto donde se lanzan, lo que hace explicita la logica de errores.

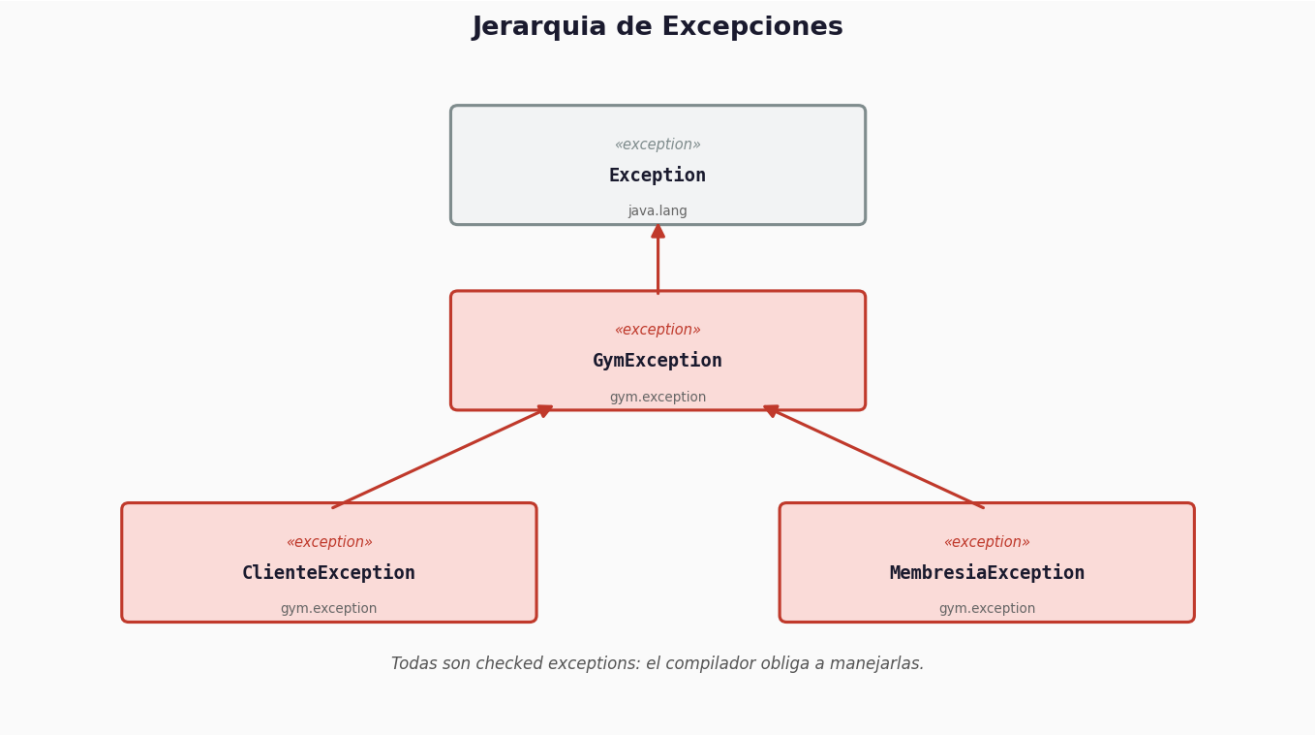

Figura 4 - Jerarquia de excepciones del dominio

Uso tipico

```
  // En AccesoController - lanzar
if (cliente.getMembresiaActiva() == null)
    throw new GymException("No tiene membresia activa.");

// En AccesoView - capturar
try {
    controller.registrar(cliente, tipo);
} catch (GymException e) {
    AlertUtil.error("Error de acceso", e.getMessage());
}
```

7. Patrones de diseno aplicados

Singleton

DataService, NotificacionService y ThemeService usan Singleton. Garantizan una unica instancia durante toda la ejecucion, evitando inconsistencias al tener multiples copias de los datos en memoria.

```
  public static DataService getInstance() {
    if (instance == null) instance = new DataService();
    return instance;
}
```

MVC (Model-View-Controller)

La vista nunca accede al modelo directamente, siempre pasa por el controlador:

```
  ClienteView  ->  ClienteController  ->  DataService  ->  Cliente
```

Componente generico SearchableTable<T>

Encapsula el patron FilteredList + SortedList + TextField para que cada vista solo declare sus columnas y su predicado de busqueda, sin repetir codigo.

```
  searchableTable.setItems(ds.getClientes(), (c, texto) ->
    c.getNombre().toLowerCase().contains(texto) ||
    c.getEmail().toLowerCase().contains(texto)
);
```

Task<T> para concurrencia en JavaFX

BackupService extiende Task<String> en lugar de Runnable. Esto permite reportar progreso con updateProgress(), mostrar mensajes con updateMessage(), y recibir callbacks en el hilo de JavaFX con setOnSucceeded().

```
  public class BackupService extends Task<String> {
    @Override
    protected String call() throws Exception {
        updateProgress(0, 4);
        // trabajo en hilo secundario sin bloquear la UI
        return filename;
    }
}
```

8. Persistencia de datos

El sistema serializa y deserializa todo el estado en un archivo gym_data.dat usando serializacion nativa de Java. El record GymData agrupa todas las listas en un unico objeto serializable.

```
  record GymData(
    List<Cliente>         clientes,
    List<ClaseGrupal>     clases,
    List<Equipo>          equipos,
    List<Recompensa>      recompensas,
    List<Pago>            pagos,
    List<RegistroAcceso>  accesos
) implements Serializable {}
```

Flujo de carga y guardado

```
  Al iniciar:
  GymApp.start()
    DataService.cargarDatos()
      Serializer.deserializar('gym_data.dat')
        OK  -> restaura clientes, clases, equipos, pagos, accesos
        FAIL -> carga datos de ejemplo automaticamente

Al cerrar la ventana:
  stage.setOnCloseRequest(...)
    DataService.guardarDatos()
      Serializer.serializar(gymData, 'gym_data.dat')
```

9. Diagrama de secuencia - Registro de acceso

Muestra el flujo completo cuando un usuario registra la entrada de un cliente. La validacion es secuencial: primero membresia, luego estado de acceso del dia.

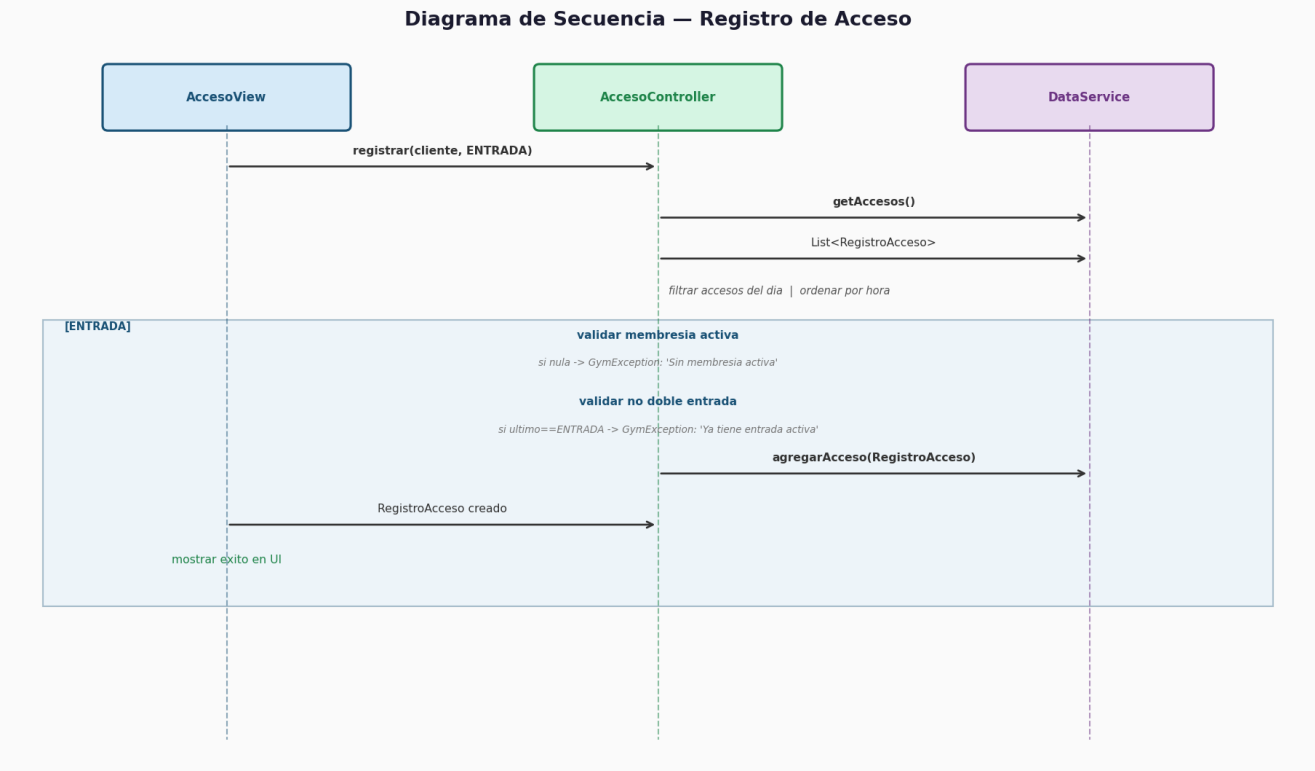

Figura 5 - Secuencia de registro de acceso con validaciones

10. Diagrama de secuencia - Backup en segundo plano

BackupService se ejecuta en un hilo daemon (muere cuando se cierra la app). El progreso se reporta en tiempo real y el callback de exito se ejecuta automaticamente en el hilo de JavaFX sin necesidad de Platform.runLater().

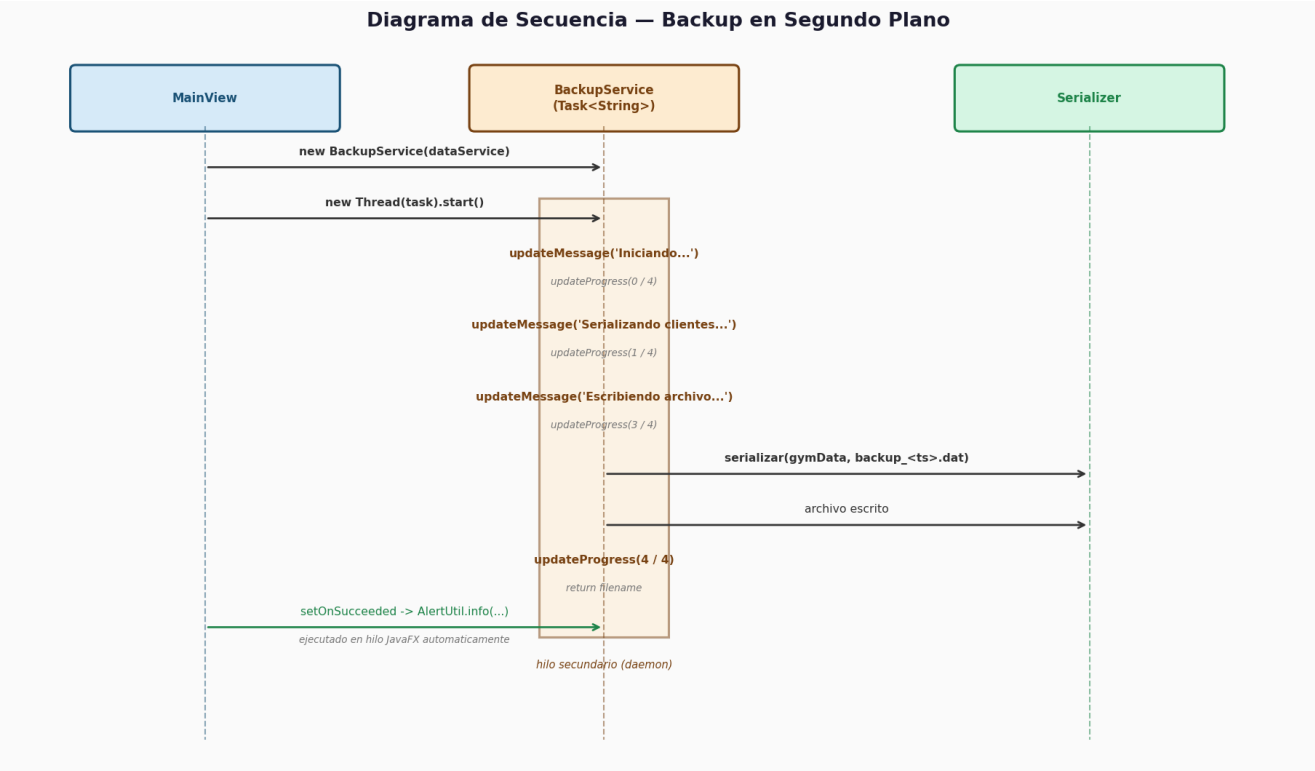

Figura 6 - Secuencia del proceso de backup con multithreading

11. Componentes de interfaz reutilizables

SearchableTable<T>

Componente que extiende VBox y combina un TextField de busqueda con un TableView. Internamente usa FilteredList<T> para filtrar en tiempo real sin modificar la lista original, y SortedList<T> para permitir ordenamiento por columna.

ConfirmDialog

Ventana modal con initModality(APPLICATION_MODAL) que bloquea la ventana padre. Soporta atajos: Enter confirma, Escape cancela. Hereda el tema activo via ThemeService.

StatusBadge

Etiqueta con color segun el estado del objeto (ACTIVA, VENCIDA, EN_MANTENIMIENTO). El color se aplica mediante clases CSS definidas en styles.css y styles-light.css.

12. Requisitos y ejecucion

Componente	Version minima
Java JDK	17 (compilado en 17, compatible con 21+)
Maven Wrapper	incluido en el repo (mvnw / mvnw.cmd)
JavaFX SDK	21 (descargado automaticamente por Maven)
Sistema operativo	Windows (natives win incluidas en pom.xml)

Comandos para clonar y ejecutar

```
  git clone https://github.com/EliasBS76/LAB_POO_2026.git
cd LAB_POO_2026/practica_11

# Windows
mvnw.cmd javafx:run

# Mac / Linux
./mvnw javafx:run

# Generar JAR ejecutable
mvnw.cmd package
java -jar target/gym-manager-1.0.jar
```

El Maven Wrapper descarga Maven 3.9.6 automaticamente. Maven descarga JavaFX 21 y OpenPDF de internet en la primera ejecucion. Si gym_data.dat no existe, el sistema carga datos de ejemplo al arrancar.